

Out of Distribution
Labs

Repository context through a portable intuition MCP

A small paired class-generation experiment

Out of Distribution Labs

October 7, 2026

Development study / Adapted RepoClassBench

Two tasks; measured agent outcomes, not a leaderboard submission.

<https://outofdistributionlabs.github.io/>

Abstract

We test whether a revision-scoped source-evidence MCP can assist repository-aware class generation. A storage-blocked feature-generation evaluation is replaced by two preselected tasks from RepoClassBench, preserving the earlier infrastructure failures as separate records. Native Codex, Codex with flat lexical retrieval, and Codex with graph-assisted retrieval receive identical masked repositories and a five-minute wall limit per attempt. Hidden tests are evaluated in separate containers; reference and empty-class controls qualify the functional scorer before inference. The contribution is a small, auditable paired experiment with real agent execution. The final section gives measured outcomes. Two convenience-selected tasks cannot establish benchmark-wide improvement, current state-of-the-art inadequacy, or usefulness to hundreds of agents.

Contents

1 Problem and testable claim	2
2 Primary sources and problem fit	2
3 Selection, runtime and controls	2
4 Information boundaries and MCP treatment	3
5 Execution and reusable research process	4
6 Interpretation and next experiment	4
7 Measured results	6

1 Problem and testable claim

Repository class generation requires local interfaces to agree with a public specification: constructor defaults, inheritance, validation, mutable behavior, and calls into existing modules. An isolated implementation can look plausible while violating these contracts. The practical decision problem is which repository evidence to inspect before writing the missing class.

Let $Y_{ia} \in \{0, 1\}$ denote all-required-tests success for task i under arm a . The broader thesis is that bounded abstraction-assisted evidence improves success per available agent budget. Here the observable paired contrasts are

$$\hat{\Delta}_{CA} = \frac{1}{2} \sum_{i=1}^2 (Y_{iC} - Y_{iA}), \quad \hat{\Delta}_{CB} = \frac{1}{2} \sum_{i=1}^2 (Y_{iC} - Y_{iB}).$$

These are descriptive development outcomes, not estimates precise enough to support a five-percentage-point population claim. A strict feasibility criterion is that isolated inference completes, the tool treatment actually consults the MCP, and a qualified evaluator can score every generated class. Effectiveness remains a separate gate.

2 Primary sources and problem fit

RepoClassBench’s primary repository describes 287 class-generation tasks: 130 Java, 97 Python and 60 C# tasks. Its associated 2024 paper proposes Retrieve-RepoTools-Reflect, already demonstrating that repository tools and iterative reasoning are relevant prior work[1]. Our MCP interface and shallow reranker do not establish a new theory of repository reasoning or supersede that work.

We inspected the pinned public task metadata, repository archive, Python masking and evaluator implementation, license, and expected-test scoring logic. Source commit and complete SHA256 values are published in the manifest. The source is MIT licensed; the third-party Pydicom and Requests repository licenses remain applicable. We publish task identifiers and sanitized measurements, not the repository archive or hidden class bodies.

Our earlier RepoQA assay measured source retrieval rather than generation and found no convincing graph advantage[3]. That result is retained. FeatureBench is a more direct test of cross-module feature contracts, but its container storage exceeded this workspace. No FeatureBench infrastructure failure is counted as a failed agent solution. The present tasks are smaller and useful for checking functional integration, but do not test large multi-file feature implementation.

Current-SOTA inadequacy on this selected RepoClassBench population has *not* been demonstrated. Historical benchmark claims and a separate FeatureBench leaderboard audit cannot supply that missing comparison. A future study needs a current baseline, adequate task coverage, and a blinded failure taxonomy before making the user’s broader SOTA claim.

3 Selection, runtime and controls

The two repository groups were chosen for small, pure-Python runtimes. Within each group we fixed the task with the shortest public detailed description, before agent outcomes. Pydicom’s Sequence task

has 13 required test IDs; Requests' obfuscated rEQueSt task has 44. The selection favors tractability and is neither random nor representative. The pinned archive contains obfuscated symbols; substituting the public upstream repository would change the task.

The runtime uses a pinned Python 3.11 slim base with 516 MB of installed runtime and search tools. An immutable inventory records direct and transitive dependencies. Pydicom requires extra test data during collection; the evaluator has fixture files downloaded from its published URL register and checked against the published hashes for downloaded fixtures. Setup-time Docker mount and filesystem-permission failures were corrected before agent execution.

Functional qualification is explicit: reference implementations pass 13/13 and 44/44 required tests. Empty classes pass 1/13 and 0/44, resolving neither task. Missing IDs, skipped tests, collection errors and failed tests cannot be treated as passed. We retain imports when masking classes, replace the official Conda environments, withhold iterative hidden-test and linter feedback, and use the official functional all-required-tests-pass rule. These deviations make this an *adapted* pilot, not an official leaderboard run.

4 Information boundaries and MCP treatment

Inference receives only masked repository source and the public detailed class specification. Git history, test paths and the unique reference class body are removed; bytecode is excluded. The source directory is read-only. The agent writes a candidate to a separate answer directory. A separate evaluator copies the original repository and inserts the sealed candidate before running the required tests.

Docker inference containers do not mount the evaluator metadata, hidden tests, host workspace or Docker socket. They mount the selected authentication file, CLI binaries, public CA trust and a restricted proxy socket. The proxy permits only the model's API endpoints; a benchmark-site request must be denied, and all ordinary container networks are detached. Runtime receipts record those checks. Authentication is never packaged into images or public research artifacts. The container retains DAC_OVERRIDE for access to selected mounted files; this is a practical information boundary, not a proof against every malicious action or model pretraining contamination.

The MCP exposes status, query and bounded evidence. The engine extracts source entities, scores lexical relevance, and in graph mode mixes file/module means and conservative local call-name hints with entity relevance. This is a shallow contextual reranker, not a semantic contract reasoner or an ontology completeness guarantee. The frozen implementation and corpus are identical between flat and graph arms.

Both tool arms use the same portable shell adapter, which starts the real stdio MCP and calls it through the official SDK. Native CLI MCP startup requires explicit Python import-path configuration. Calls through the adapter are independently logged. Tool-arm prompts require status and at least one query, then allow evidence calls. Consequently native versus tool measures the whole tool-plus-instruction treatment; graph versus flat controls that instruction difference. Per-call process startup and repeated indexing belong to the measured wall cost.

5 Execution and reusable research process

Each of six scheduled runs uses a fresh ephemeral Codex session and container, model gpt-6.1-sol at medium reasoning effort, a 300-second inference ceiling, and one attempt. Task order is fixed; arm order is randomized separately with seeds 20261007 and 20261008. Frozen artifact hashes include the engine, server, adapters and runner. No outcome-driven engine tuning or task replacement is allowed. The first native Pydicom attempt was interrupted by an account usage limit before producing a candidate. Following an explicit user request to try again and a successful access probe, that cell was retried unchanged. This is a disclosed post-interruption deviation from the original six-attempt ceiling: six planned scored cells may involve seven inference attempts. The interrupted attempt is not a solution failure.

The reusable process has six stages: protocol/selection; runtime/scorer qualification; hidden-answer isolation and MCP readiness; frozen execution; analysis; paper/publication. The expensive stages split into dependency setup, positive/negative controls, network checks, real tool calls, inference, hidden grading and accounting. Each stage ends in an advance/redo/stop sentinel with evidence. Reviews here are author self-review, not independent review. The longer twelve-stage methodology remains appropriate for a powered confirmatory study[2].

Progress pages update every 60 seconds while active; notifications use a separate configurable 300-second cadence. Commits preserve each checkpoint. Results record required-test counts, whole-task success, wall time, input/output/cached token counters, actual tool calls, source hashes and errors. Monetary cost is unknown and reported as null. Caching is observed rather than assumed absent; cumulative input counters are not unique prompt lengths.

6 Interpretation and next experiment

The chosen tasks can expose obvious implementation or integration failures, but two tasks cannot separate chance, task difficulty and treatment effect. A single model, one attempt per cell, convenient selection, retained imports, altered environments and possible prior exposure constrain every comparison. Success on all required tests does not imply full-program correctness or performance on unrelated regressions.

A useful next experiment stratifies tasks by dependency depth, interface ambiguity and invariant burden using blinded source inspection, then reserves repositories for evaluation. Compare the same flat and richer contract-aware engines at matched total budgets, with multiple randomized attempts and honest failure accounting. Use official environments before external leaderboard comparison. The contract engine should justify its added information on failure categories that a strong current baseline actually exhibits, rather than on theoretical appeal alone.

References

- [1] Microsoft Research. Repoclassbench: Natural language to class-level code generation by iterative tool-augmented reasoning over repository. <https://github.com/microsoft/repoclassbench>, 2024. Primary code and evaluator inspected at a347b55; associated paper arXiv:2405.01573.

- [2] Out of Distribution Labs. Intuition engine: benchmarking methodology. <https://outofdistributionlabs.github.io/research/intuition-engine-benchmarking/progress.html>, 2026.
- [3] Out of Distribution Labs. Testing an intuition engine for repository decisions. <https://outofdistributionlabs.github.io/research/intuition-engine-pilot/white-paper.pdf>, 2026.

7 Measured results

All six planned scored cells completed. One initial native Pydicom attempt was interrupted by an account usage limit and retried unchanged; it is infrastructure-invalid and excluded from solution scoring.

Task	Native	Flat MCP	Graph MCP
Pydicom Sequence	10/13; fail	10/13; fail	10/13; fail
Requests rEQueSt	44/44; pass	44/44; pass	44/44; pass
Whole tasks resolved	1/2	1/2	1/2

Both paired resolution contrasts are zero. There is no measured success advantage for the graph engine in this sample.

Arm	Seconds	Input	Cached input	Output	Operations
A	90.00	213,202	176,384	3,327	0
B	108.53	192,451	158,848	3,822	6
C	109.26	191,729	169,088	3,503	6

A is native, B flat and C graph. Times and token counters are totals over the two valid scored cells per arm. They exclude container setup, readiness probes and the interrupted attempt, whose token spend is unknown. Dollar cost remains null. Input counters accumulate across turns and include caching; they are not unique prompt lengths. Graph and flat take about 21% longer in total than native here; two task timings do not establish an efficiency effect.

Each tool cell logs status, query and evidence operations. The shell adapter obtains a snapshot through an additional status call for query/evidence: three logged operations correspond to five underlying MCP tool requests. All logged responses succeed. No tool consultation is inferred merely from configuration.

All three Pydicom attempts fail the same three exact exception-message checks, despite raising the requested exception types. This is an incidental error-message contract mismatch, not evidence that the agent cannot reason about deep dependencies. The required tests are not softened after seeing outcomes. All three Requests candidates satisfy all 44 required tests.

The supported result is successful isolated integration and a tied, negative-effectiveness pilot. It does not show current SOTA inadequacy, a graph advantage, or large-agent scaling. The next experiment should identify substantive dependency/invariant failures in a strong baseline, then freeze a broader official-environment evaluation with sufficient precision.